InverseCoder: Self-improving Instruction-Tuned Code LLMs with Inverse-Instruct

Yutong Wu[1, 2], Di Huang[1], Wenxuan Shi[1, 2], Wei Wang[3], Lingzhe Gao[3]
Shihao Liu[3], Ziyuan Nan[1, 2], Kaizhao Yuan[1, 2], Rui Zhang[1], Xishan Zhang[1],
Zidong Du[1], Qi Guo[1], Yewen Pu[4], Dawei Yin[3], Xing Hu[1], Yunji Chen[1, 2*]
[1]SKL of Processors, Institute of Computing Technology, CAS
[2]University of Chinese Academy of Sciences
[3]Baidu Inc., Beijing, China
[4]Autodesk Research

arXiv:2407.05700v2 [cs.CL] 16 Dec 2024

Abstract

Recent advancements in open-source code large language models (LLMs) have been driven by fine-tuning on the data generated from powerful closed-source LLMs, which are expensive to obtain. This paper explores whether it is possible to use a fine-tuned open-source model to generate additional data to augment its instruction-tuning dataset. We make two observations: (1) A code snippet can serve as the response to different instructions. (2) Instruction-tuned code LLMs perform better at translating code into instructions than the reverse. Based on these observations, we propose Inverse-Instruct, a data augmentation technique that uses a fine-tuned LLM to generate additional instructions of code responses from its own training dataset. The additional instruction-response pairs are added to the original dataset, and a stronger code LLM can be obtained by fine-tuning on the augmented dataset. We empirically validate Inverse-Instruct on a range of open-source code models (*e.g.*, CodeLlama-Python and DeepSeek-Coder) and benchmarks (e.g., HumanEval(+), MBPP(+), DS-1000 and MultiPL-E), showing it consistently improves the base models.

Code — https://github.com/wyt2000/InverseCoder
Extended version — https://arxiv.org/abs/2407.05700

1 Introduction

Code generation, which aims to generate code that satisfies the user's intent from inputs/outputs or natural language, has been a significant challenge in computer science. Recently, closed-source LLMs like GPT-3.5 and GPT-4 (OpenAI 2023) have enabled the generation of general-purpose code (like Python) based on natural language, making them broadly applicable in the fields of programming assistance (Microsoft 2023), computer vision (Surís, Menon, and Vondrick 2023; Gupta and Kembhavi 2023), science (Nejjar et al. 2023), and embodied intelligence (Liang et al. 2023; Ma et al. 2023; Tang, Key, and Ellis 2024; Wang et al. 2023).

To develop high-performance open-source models, researchers have leveraged these closed-source LLMs to generate datasets of instructions and code, then distilled these datasets into smaller, open-source code LLMs via instruction tuning (Luo et al. 2023; Wei et al. 2023; Yu et al. 2023; Song et al. 2024). For example, Code Alpaca (Chaudhary 2023) was fine-tuned on 20K instruction-code pairs generated based on GPT-3.5 with SELF-INSTRUCT (Wang et al. 2022). Luo et al. (2023) used *Evol-Instruct* (Xu et al. 2023), a method that creates a diverse set of instruction data from GPT-3.5 for code generation via evolution heuristics. OSS-INSTRUCT (Wei et al. 2023) first creates coding problems from the source code snippet, then queries strong LLMs for their corresponding solutions. Fine-tuned with 75K GPT-3.5 OSS-INSTRUCT data and 110K GPT-4 *Evol-Instruct* data (*i.e.* `evol-codealpaca-v1`) (theblackcat102 2023), Magicoder$\mathcal{S}$ series achieve state-of-the-art (SOTA) results among open-source code models. These approaches have one thing in common: they heavily rely on generating data by querying stronger closed-source LLMs (*e.g.*, GPT-4), which incurs significant additional expenses. Therefore, it is crucial to develop a self-improvement method for open-source models without relying on stronger guidance.

This paper explores how to improve an instruction-tuned code LLM by *querying itself* (rather than querying a closed-source LLM). We make the following two observations :

1. A single code snippet can serve as a valid response to multiple instructions.
2. Instruction-tuned code LLMs perform better at translating code into instructions than translating instructions into code (see Section 3).

The first observation suggests that an instruction-tuned LLM can generate a new instruction for each response code in its training dataset, thereby expanding the original dataset. The second observation confirms that generating data in this way (Code-to-NL) is more effective than NL-to-Code.

Therefore, we develop Inverse-Instruct, a simple yet effective instruction tuning approach based on self-generating instructions from code snippets (Figure 1). Inverse-Instruct starts with an instruction-code corpus, and a code LLM fine-tuned on it. We first clean and extract code snippets from the corpus, then let the code LLM translate these code snippets into new instructions. Next, we use the code LLM to evaluate and filter consistent instruction-code pairs from the newly generated data. Finally, the filtered dataset is combined with the original instruction dataset to fine-tune a new model. The main differences between Inverse-Instruct and previous data generation methods are discussed in Section 2.2.

*Corresponding author.
Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

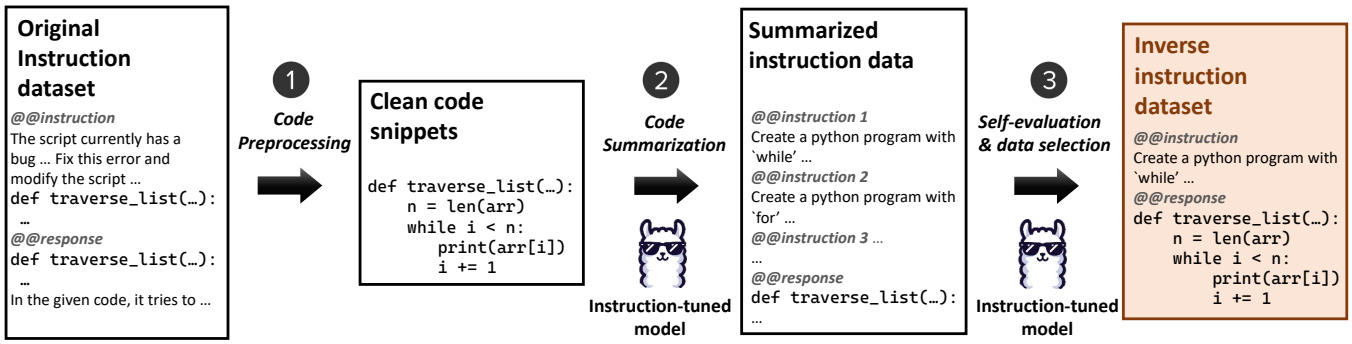

Figure 1: **The overview of Inverse-Instruct.** Inverse-Instruct utilizes the models' own capability in code summarization to generate an inverse instruction dataset which can further enhance the model's performance. Inverse-Instruct consists of three steps, including code preprocessing, code summarization, and self-evaluation & data selection.

Using Inverse-Instruct, we develop InverseCoder, a series of fine-tuned code LLMs that achieve SOTA results. We evaluated InverseCoder on a wide range of benchmarks (Section 6), including HumanEval(+) (Chen et al. 2021; Liu et al. 2023), MBPP(+) (Austin et al. 2021; Liu et al. 2023), MultiPL-E (Cassano et al. 2023), and DS-1000 (Lai et al. 2023). Results show that InverseCoder series surpasses the base models by exploiting the base models' own capability. Specifically, InverseCoder-DS-6.7B achieves 76.8% on HumanEval+, 69.0% on MBPP+, 62.6% on MultiPL-E, 44.2% on DS-1000, which are SOTA results across four benchmarks among fully open-source (both model and dataset) models with only 6.7B parameters.

Our key contributions are introducing Inverse-Instruct, an effective self-improvement instruction tuning approach for code LLMs and presenting a series of code LLMs named InverseCoder, which achieves SOTA or comparative results on a wide range of benchmarks.

We organize the structure of the paper as follows: Section 2 introduces related works. Section 3 shows the evidence of our observations. Section 4, 5 provide a detailed explanation of our approach (*i.e.* Inverse-Instruct). Section 6 presents the experiments for our models (*i.e.* InverseCoder). Section 7 concludes with a summary.

2 Related Work

2.1 LLMs for Code Generation

After being pre-trained on a large amount of code, LLMs have demonstrated impressive code generation capabilities. Recently, AI code assistants have become one of the most important applications of LLMs. Technology companies such as OpenAI and Google have developed and released many closed-source large language models, including Codex (Chen et al. 2021), GPT-4 (OpenAI 2023), PaLM (Chowdhery et al. 2022), and Gemini (Team et al. 2023), which have achieved outstanding performance on code generation benchmarks.

In addition to closed-source models, there are also some available open-source models whose weights and training data are available to the public, such as CodeGen (Nijkamp et al. 2022), CodeGeeX (Zheng et al. 2023), AlphaCode (Li et al. 2022), CodeT5 series (Wang et al. 2021), StarCoder series (Li et al. 2023; Lozhkov et al. 2024), CodeLlama (Rozière et al. 2023), DeepSeek-Coder (Guo et al. 2024) and CodeQwen (Bai et al. 2023). These open-source code models have shown notable advancements in code-related tasks, but there is still a gap compared to the most advanced code LLMs.

2.2 Instruction-Tuned Code LLMs

Instruction tuning is a method for further enhancing the instruction-following capability of pre-trained LLMs. It has been widely applied to the LLMs for general tasks including T5 (Raffel et al. 2020) and FLAN (Wei et al. 2021).

For code LLMs, OctoPack (Muennighoff et al. 2023) and PIE (Shypula et al. 2024) extracted high-quality data from human-written instructions and code. Fine-tuning with these data has significantly enhanced the program generation capabilities of the base models.

However, obtaining high-quality human-written instruction datasets is usually laborious. Researchers have attempted to employ stronger closed-source LLMs to generate both new instructions and responses for instruction-tuning. Specifically, CodeAlpaca (Chaudhary 2023) sampled tasks from a seed task pool and prompted a stronger LLM to generate instruction-tuning data based on the seed tasks. WizardCoder (Luo et al. 2023) prompted a stronger LLM to generate more complex instructions and the corresponding responses. Magicoder (Wei et al. 2023) used a stronger LLM to create problems and code solutions based on open-source code snippets, as the seed code snippets offer controllability and diversity to the generation. WaveCoder (Yu et al. 2023) used a stronger LLM to both generate and discriminate the instruction-response pair for different coding tasks (*e.g.*, code summarization and code repair). AlchemistCoder (Song et al. 2024) employed a stronger LLM to add more details for existing instructions.

The main differences between our method and the aforementioned related works are:

- We focus on the **self-improvement** of open-source code models rather than relying on stronger guidance (such as human annotation or advanced LLMs like GPT-4).

Generation Method	WC-CL-7B	WC-DS-6.7B
NL $\rightarrow$ Code	62.4	70.2
Code $\rightarrow$ NL $\xrightarrow{\text{GPT-4}}$ Code	**74.3**	**79.0**
Code $\rightarrow$ NL $\xrightarrow{\text{Humans}}$ Code	**86.7**	**80.0**

Table 1: **Pass@1 (%) results on MBPP+ in observation checking experiments.** The abbreviations "WC-CL-7B" and "WC-DS-6.7B" refer to the instruction-tuned models WizardCoder-GPT4-CL and WizardCoder-GPT4-DS. Line 1 represents the evaluation of NL-to-Code for instruction-tuned open models. Lines 2 and 3 evaluate Code-to-NL by leveraging GPT-4 and humans to convert NL into its equivalent code, then assess its correctness against the original code. We removed the problems that GPT-4 was unable to give executable code for them.

- We generate new data by **converting code to instructions** from existing datasets rather than generating code from instructions.

3 Sanity Check: Code-to-NL vs. NL-to-Code

In this section, we validate our observation that instruction-tuned code LLMs perform better at translating code into instructions (*i.e.*, Code-to-NL) than translating instructions into code (*i.e.*, NL-to-Code) through an experiment.

We first select a manually written set of correctly matched NL-Code pairs $\{x, y\}$ with unit tests and prompted a fine-tuned code LLM to convert x into new code y' and y into new NL x' separately. Then, We use the following metrics to quantify the model's performance in the two tasks:

- For NL-to-Code, we use unit tests to evaluate the functional correctness of generated code y' against original code y.
- For Code-to-NL, we convert generated NL x' to an equivalent code snippet $\hat{y}$ by humans and a stronger code LLM. Then we measured the functional correctness of $\hat{y}$ by unit tests.

Specifically, we use the problem-answer pairs with unit tests in a basic Python generation benchmark MBPP+ (Liu et al. 2023) as matched NL-Code pairs $\{x, y\}$. For NL-to-Code, we took all 378 problems in the benchmark for evaluation. For Code-to-NL, we first select 30 problems for humans to write the equivalent code of the generated NL, and then we employ GPT-4 to finish this task for all problems.

We evaluate two instruction fine-tuned code LLMs (*i.e.*, WizardCoder-GPT4-CL and WizardCoder-GPT4-DS, which are instruction-tuned by 110K GPT-4 dataset `evol-codealpaca-v1`). The results are shown in Table 1. From the table, we conclude that (Code $\rightarrow$ NL) is better than (NL $\rightarrow$ Code), showing that code LLMs perform better in code summarization than in code generation.

4 Inverse-Instruct: Data Augmentation via Code Summarization

In this section, we will introduce Inverse-Instruct, a data augmentation method that can obtain more instruction data through the model's own capabilities. The overall illustration of Inverse-Instruct is shown in Figure 1. Inverse-Instruct is founded on the following two observations: (1) The same code can be considered as a response to different instructions, which expands the dataset effectively. (2) Converting formal language (i.e., code) into informal language (i.e., natural language) is generally more straightforward than the reverse.

The whole data generation process contains three stages: (1) Code preprocessing. (2) Code summarization. (3) Self-evaluation and data selection. In code preprocessing, we preprocess the code data by filtering clean code snippets $\{y_i^*\}$ from an off-the-shelf instruction tuning dataset $\{(x_i, y_i)\}$ (*e.g.*, `evol-codealpaca-v1`). Subsequently, in code summarization, we prompt an instruction fine-tuned code LLM M (*e.g.*, WizardCoder-GPT4-CL) to convert the clean code snippets $\{y_i^*\}$ to multiple new instructions $\{x_{ij}^*\}$. Then, in self-evaluation and data selection, we use the same code LLM M to select the best instruction x_i^{**} in $\{x_{ij}^*\}$. The selected instructions $\{x_i^{**}\}$ are combined with the original code snippets $\{y_i^*\}$ to construct a new instruction tuning dataset $\{(x_i^{**}, y_i^*)\}$. Finally, we fine-tune the base code LLM with the instruction data $\{(x_i^{**}, y_i^*)\} \cup \{(x_i, y_i)\}$ to obtain a stronger code LLM (*i.e.* InverseCoder). Details of the three steps are illustrated below.

4.1 Code Preprocessing

The first step is to preprocess the existing code data and get clean code snippets $\{y_i^*\}$. This is because the Code-to-NL capabilities of code LLMs can only be fully utilized with clean code, whereas the response data $\{y_i\}$ in the original dataset typically contains a lot of noise, such as natural language responses.

We select data with code snippet $\{y_i^*\}$ from the original $\{y_i\}$ with the following two steps:

1. **Filtering responses**. We first collect responses that contain the marker of the code block (*i.e.* ```), which indicates that there are code snippets in the response. The remaining data might contain clean code without any code markers, so then we collect the responses that can pass syntax checking.
2. **Extracting code**. After filtering responses with code snippets, we remove the natural language surrounding the code snippets to make it easier for the model to summarize. If there are multiple parts of code in the original response, we only keep the first part, since the following parts are usually test cases or using examples.

At the end of code preprocessing, we obtain clean code snippets $\{y_i^*\}$ for summarization.

4.2 Code Summarization

After filtering, we employ the code LLM M to generate a certain number of corresponding instructions $\{x_{ij}^*\}$ for each code snippet in $\{y_i^*\}$ by summarizing its functionality. During the summarization process, we randomly choose different instruction prefixes for the prompt to enhance the diversity of the instructions.

In this way, we have obtained new pairs of natural language and code $\{(x_{ij}^*, y_i^*)\}$.

4.3 Self-evaluation and Data Selection

We noticed that code LLM M might make mistakes during the code summarization process. Therefore, we utilize M itself to evaluate $\{(x_{ij}^*, y_i^*)\}$ and select the most appropriate instruction.

Data selection is typically performed by powerful LLMs such as GPT-4 because these models possess excellent instruction-following capabilities, enabling them to understand complex filtering rules (Wang et al. 2024a). However, the instruction-following capabilities of code LLMs are often weaker, making it difficult to conduct effective selection. (See the comparison experiments in Section 6.5).

Inspired by AutoMathText (Zhang et al. 2024), we use the pseudo-probability of `YES` token given by the code LLM M as an indicator of the instruction quality rather than a score in textual format. Specifically, we concatenate the generated instructions $\{x_{ij}^*\}$ and the original code snippets $\{y_i^*\}$ as problem-answer pairs $\{(x_{ij}^*, y_i^*)\}$. Then, we ask M to evaluate the correctness of each answer under the given problem and calculate the pseudo-probability of `YES` using the logits of the first token given by M. The formula for calculating the pseudo-probability is shown as follows (Zhang et al. 2024):

$$\text{LM-Score}(\cdot) = \frac{\exp(\text{logit}(\text{`YES'}))}{\exp(\text{logit}(\text{`YES'})) + \exp(\text{logit}(\text{`NO'}))}$$

After evaluation, we select the instruction with the highest score x_i^{**} for each response in $\{y_i^*\}$ to obtain a new training dataset $\{(x_i^{**}, y_i^*)\}$.

5 Implementation Details

The original instruction tuning dataset. In this work, we mainly use `evol-codealpaca-v1` as our original instruction tuning dataset $\{(x_i, y_i)\}$, which is widely used for instruction tuning of code LLMs (Wei et al. 2023; Yu et al. 2023; Song et al. 2024). It contains 111183 instruction-response pairs generated by *Evol-Instruct* using GPT-4. Following Magicoder (Wei et al. 2023), `evol-codealpaca-v1` is decontaminated by removing data that contain docstrings or solutions from HumanEval (Chen et al. 2021), MBPP (Austin et al. 2021), MultiPL-E (Cassano et al. 2023), and DS-1000 (Lai et al. 2023), which are used to evaluate InverseCoder. We apply the same decontamination method to the newly generated instruction data $\{(x_i^{**}, y_i^*)\}$.

Training for original Code LLM. We take CodeLlama-Python-13B, CodeLlama-Python-7B (Rozière et al. 2023) and DeepSeek-Coder-Base-6.7B (Guo et al. 2024) as base models. To obtain the beginning code LLM M (hereinafter called WizardCoder-GPT4), we fine-tune the base models on `evol-codealpaca-v1` for 2 epochs using 8 NVIDIA A100-40GB SMX GPUs. We set the initial learning rate at $5e-5$ with 15 warmup steps and a linear learning rate scheduler. We use Adafactor (Shazeer and Stern 2018) as our optimizer and choose a batch size of 512 with a sequence truncation length of 1024.

Instruction data collection. We use the vLLM inference framework (Kwon et al. 2023) for code summarization and instruction selection on the same GPUs as training. We generate 10 instructions $\{x_{ij}^*\}_{j=1}^{10}$ for each code snippet in the code summarization stage. For each instruction-response pair, the self-evaluation and data selection process is conducted by prompting the beginning code LLM M with greedy decoding. We choose the instruction with the highest pseudo-probability of `YES` as the best-generated instruction for each response.

Training for InverseCoder. Following Magicoder$\mathcal{S}$ (Wei et al. 2023), we first fine-tune the base models on the new dataset $\{(x_i^{**}, y_i^*)\}$ with 90363 instruction-response pairs (generated by the original Code LLM M) for 1 epoch, then we continue to fine-tune the models with the original dataset $\{(x_i, y_i)\}$ (generated by GPT-4) for 2 epochs to obtain InverseCoder. The hyperparameters are the same as the training process for the original code LLM M. The instruction tuning prompt is aligned with Magicoder$\mathcal{S}$.

6 Experiments

We conduct a series of experiments to investigate these topics:

1. InverseCoder's performance on benchmarks (Sec. 6.1).
2. Impact of each stage in Inverse-Instruct (Sec. 6.2).
3. Impact of dataset size scaling (Sec. 6.3).
4. Is Inverse-Instruct effective on other datasets (Sec. 6.4)?
5. Comparison with other data selection methods (Sec. 6.5).
6. Does selecting multiple self-generated instructions for each response lead to further improvement (Sec. 6.6)?
7. Can Inverse-Instruct be repeatedly applied to InverseCoder to achieve multi-round optimization (Sec. 6.7)?
8. Can Inverse-Instruct be further optimized by using additional self-generated code as responses (Sec. 6.8)?

6.1 Main Results

We train InverseCoder on three base models with different parameter sizes and evaluate them on four benchmarks widely used for code LLMs, including Python text-to-code generation, multilingual coding, and data-science code generation. The results show that **the performance of SOTA code LLMs can continue to improve by Inverse-Instruct**.

Baselines. We compare the performance of our models with a wide range of baselines including:

1. **Base Models**: Three base models mentioned in Section 5. We compare InverseCoder with them to show the absolute improvement of the whole instruction-tuning process.
2. **WizardCoder-GPT4**: The beginning code LLMs in our data generation process, which are only trained by the original instruction-tuning dataset (*i.e.*, `evol-codealpaca-v1`). We compared InverseCoder with them to show the improvement brought by Inverse-Instruct.
3. **Other Open Source Instruction-Tuned Code LLMs**: Instruction-tuned code models in related works, including Magicoder$\mathcal{S}$ (Wei et al. 2023), WaveCoder-Ultra-DS (Yu et al. 2023) and AlchemistCoder (Song et al.

Model	Common Data	Specific Data
WizardCoder-GPT-4		0K (baseline)
Magicoder$\mathcal{S}$		75K GPT-3.5
WaveCoder-Ultra	110K GPT-4	20K GPT-4
AlchemistCoder		$>$ 80K GPT-3.5
InverseCoder (ours)		90K **self-generated**

Table 2: **Training data size of different instruction-tuned code LLMs.** It is worth noting that only InverseCoder is trained by self-generated data, which is easier to obtain at a lower cost.

Model	HumanEval (+)	MBPP (+)
(Closed-source Models)		
GPT-4-Turbo (April 2024)	**90.2 (86.6)**	**85.7 (73.3)**
GPT-3.5-Turbo (Nov 2023)	76.8 (70.7)	82.5 (69.7)
(Based on CodeLlama-Python-13B)		
CodeLlama-Python-13B	42.7 (38.4)	63.5 (52.6)
WizardCoder-GPT4-CL-13B	76.8 (70.7)	73.5 (62.2)
InverseCoder-CL-13B (ours)	**79.9 (74.4)**	**74.6 (63.0)**
(Based on CodeLlama-Python-7B)		
CodeLlama-Python-7B	37.8 (35.4)	59.5 (46.8)
Magicoder$\mathcal{S}$-CL-7B	70.7 (67.7)	**70.6 (60.1)**
AlchemistCoder-CL-7B	74.4 (68.3)	68.5 (55.1)
WizardCoder-GPT4-CL-7B	72.6 (68.9)	69.3 (59.3)
InverseCoder-CL-7B (ours)	**76.2 (72.0)**	**70.6 (60.1)**
(Based on DeepSeek-Coder-6.7B)		
DeepSeek-Coder-6.7B	47.6 (39.6)	72.0 (58.7)
Magicoder$\mathcal{S}$-DS-6.7B	76.8 (71.3)	**79.4 (69.0)**
WaveCoder-Ultra-DS-6.7B	75.0 (69.5)	74.9 (63.5)
AlchemistCoder-DS-6.7B	**79.9** (75.6)	77.0 (60.2)
WizardCoder-GPT4-DS-6.7B	77.4 (73.2)	77.8 (67.5)
InverseCoder-DS-6.7B (ours)	**79.9 (76.8)**	78.6 (**69.0**)

Table 3: **Pass@1 (%) results of different LLMs on HumanEval (+) and MBPP (+) computed with greedy decoding.** The abbreviations "CL" and "DS" refer to the base models CodeLlama-Python and DeepSeek-Coder, respectively. We report other results consistently from the EvalPlus (Liu et al. 2023) Leaderboard in August 2024 and Magicoder (Wei et al. 2023) paper.

2024). They are trained on additional data generated by stronger closed-source LLMs (*e.g.*, GPT-3.5) in addition to `evol-codealpaca-v1`.

The comparison of training data size is shown in Table 2. The actual data consumption of InverseCoder should be mainly measured by the scale of the **original training dataset (110K)** since the cost of self-generating data is much lower than generating data by querying closed-source LLMs (Irugalbandara et al. 2023).

4. **Closed-source LLMs**: GPT-3.5 (OpenAI 2022) and GPT-4 (OpenAI 2023) to show the gap between InverseCoder with the advanced closed-source LLMs.

Model	Java	JS	C++	PHP	Swift	Rust	Avg.
(Based on CodeLlama-Python-13B)							
WizardCoder-GPT4*	**55.4**	64.2	55.9	52.0	49.9	53.4	55.1
InverseCoder (ours)*	54.5	**65.4**	**58.1**	**55.3**	**52.5**	**55.6**	**56.9**
(Based on CodeLlama-Python-7B)							
CodeLlama-Python	29.1	35.7	30.2	29.0	27.1	27.0	29.7
Magicoder$\mathcal{S}$ *	49.8	**62.6**	50.2	53.3	44.9	43.8	50.8
WizardCoder-GPT4*	**50.4**	60.7	50.6	51.6	45.6	**48.2**	51.2
InverseCoder (ours)*	48.7	61.9	**52.6**	**55.2**	**53.0**	46.1	**52.9**
(Based on DeepSeek-Coder-6.7B)							
Magicoder$\mathcal{S}$ *	59.6	69.8	70.0	**64.4**	**54.4**	53.6	62.0
WizardCoder-GPT4*	**61.4**	66.4	68.7	61.8	52.6	56.1	61.2
InverseCoder (ours)*	60.7	**70.1**	**70.5**	63.6	53.0	**57.4**	**62.6**

Table 4: **Pass@1 (%) results of different LLMs on MultiPL-E.** The models marked with (*) are evaluated with the same prompt format as training and the same hyperparameter as Magicoder. We report other results consistently from Magicoder paper.

Inverse-Instruct improves general Python code generation capabilities. We use HumanEval(+) and MBPP(+) (Liu et al. 2023), the enhanced versions of two Python code generation benchmarks (Chen et al. 2021; Austin et al. 2021), to evaluate the text-to-code capability of InverseCoder. Each benchmark provides a set of tasks with natural language descriptions as prompts for the code LLM to generate function-level code, which is then validated using pre-prepared test cases.

We use the pass@1 (Chen et al. 2021) score to compare the code generation capability among different models. The results are shown in Table 3, which demonstrate that InverseCoder makes a significant improvement over WizardCoder-GPT4 in Python code generation capability. Furthermore, InverseCoder-DS-6.7B has an outstanding performance in HumanEval/HumanEval+, which surpasses all open-source models with a similar scale of weights.

The improvement of Inverse-Instruct is reflected across multiple programming languages. Besides Python, we evaluate the code generation capabilities of other six mainstream programming languages for InverseCoder on MultiPL-E benchmark (Cassano et al. 2023). We generate and evaluate code of different programming languages under the inference prompt format aligned with the prompt we used in the training process.

Table 4 shows the performances of InverseCoder and other models on MultiPL-E. The results reveal that the capabilities of InverseCoder to generate code in multiple mainstream programming languages are improved over WizardCoder-GPT4.

Inverse-Instruct also leads to enhancement in data science code generation tasks. To show the capability of InverseCoder for complex programming problems in realistic applications, we evaluate it on DS-1000 benchmark (Lai et al. 2023), which comprises 1000 different data science work-

Model	plt.	np.	pd.	torch	scipy	sklearn	tf.	All
(Based on CodeLlama-Python-13B)								
WizardCoder-GPT4	**56.1**	52.2	30.3	43.0	**25.2**	49.5	40.0	42.1
InverseCoder (ours)	53.0	**54.3**	**32.1**	**50.9**	22.5	**50.5**	**43.8**	**43.1**
(Based on CodeLlama-Python-7B)								
CodeLlama-Python	55.3	34.5	16.4	19.9	22.3	17.6	28.5	28.0
WizardCoder	53.5	34.4	15.2	25.7	21.0	24.5	28.9	28.4
Magicoder$\mathcal{S}$	**55.9**	40.6	28.4	40.4	28.8	35.8	37.6	37.5
WizardCoder-GPT4	51.5	46.9	**29.9**	**43.6**	**34.9**	**41.9**	39.0	**40.2**
InverseCoder (ours)	54.2	**48.6**	27.4	38.0	34.0	**41.9**	**40.3**	39.9
(Based on DeepSeek-Coder-6.7B)								
Magicoder$\mathcal{S}$	54.8	48.9	30.0	49.2	27.3	44.7	41.2	41.2
WizardCoder-GPT4	53.8	**53.9**	28.0	49.3	**30.4**	45.7	**44.4**	42.2
InverseCoder (ours)	**55.5**	**53.9**	**32.3**	**56.7**	30.0	**50.3**	33.9	**44.2**

Table 5: **Pass@1 (%) results on DS-1000** including seven data science libraries: Matplotlib (plt.), Numpy (np.), Pandas (pd.), Pytorch, Scipy, Sklearn and Tensorflow (tf.). We evaluate our models in the same prompt and hyperparameters as Magicoder. We report other results from Magicoder paper.

Method	HumanEval(+)	MBPP(+)
Gen. + Eval.	70.7 (67.1)	**70.9** (**60.1**)
Pre.	72.6 (68.9)	69.8 (59.8)
Pre. + Sum.	75.6 (71.3)	68.0 (58.2)
Pre. + Sum. + Eval. (ours)	**76.2** (**72.0**)	70.6 (**60.1**)

Table 6: **Pass@1 (%) results on HumanEval+ and MBPP+ in ablation studies.** Preprocessing (Pre.), Summarization (Sum.) and Evaluation (Eval.) correspond to the three steps in our method. Generation (Gen.) represents regenerate responses for each instruction.

flows across seven libraries. Following Wei et al. (2023), we evaluate our model only on the completion mode.

The results in Table 5 show that the average performances of InverseCoder-CL-13B and InverseCoder-DS-6.7B in the data science code generation tasks are enhanced, which implies that Inverse-Instruct can help to improve the code generation capability of the original model in realistic tasks beyond basic programming problems.

6.2 Ablation Study

We conduct a series of ablation experiments to analyze the utility of code summarization and data selection steps in our method. We use CodeLlama-Python-7B as the base model in the following experiments with the same training settings as InverseCoder and present the results in Table 6. The ablation experiments are in three aspects:

Inverse-Instruct outperforms the NL-to-Code data generation method (Gen. + Eval.). We regenerate 10 responses $\{y_{ij}\}_{j=1}^{10}$ for each instruction x_i in the original training dataset and apply the same self-evaluation method to select the best responses. It shows that the code summarization step provides overall better performance than generating responses from instructions.

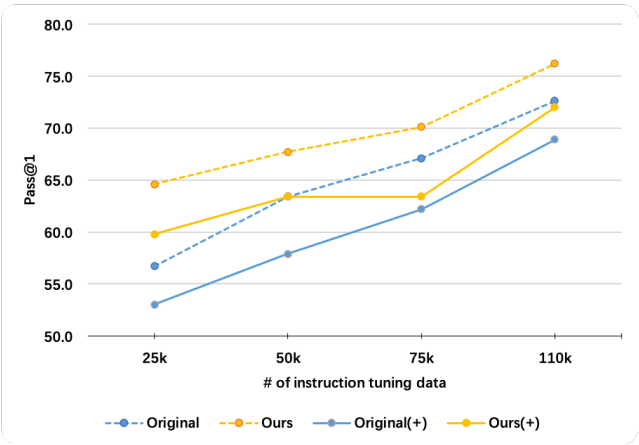

Figure 2: **Impact of data scaling.** The dashed line represents HumanEval and the solid line represents HumanEval+. Legend "Original" and "Ours" represent the original models and the models improved by Inverse-Instruct.

Model	HumanEval (+)	MBPP (+)
Magicoder-DS	66.5 (60.4)	75.4 (61.9)
InverseCoder-DS-OSS	**69.5 (64.0)**	**77.0 (66.1)**

Table 7: **Performance improvement of Inverse-Instruct when applied to Magicoder-OSS-Instruct-75K.**

Performance improvement comes not only from the preprocessing step (Pre.). We only apply preprocessing to the responses in the original dataset $\{(x_i, y_i)\}$ to obtain a cleaned dataset $\{(x_i, y_i^*)\}$. We train the models with the cleaned dataset and the original one to show the improvement from preprocessing is minor.

The self-evaluation and data selection step also plays a role in Inverse-Instruct (Pre. + Sum.). To study the role of self-evaluation and data selection, we generate only one instruction for each response in the code summarization step without any selection. The results show that self-evaluation and selection are also helpful to performance improvement.

6.3 Data Scaling

Inverse-Instruct is effective across different data scales. We conduct a series of experiments to explore the data scaling law of Inverse-Instruct. Specifically, we randomly select 25K, 50K, and 75K instruction-response pairs from the original dataset and train 3 weaker original models with them. Then, we apply Inverse-Instruct for the original models. It is shown that the performances of the models are all improved by Inverse-Instruct at different scales of data (Figure 2).

6.4 Impact of Original Dataset

Inverse-Instruct is effective across different original datasets. We apply Inverse-Instruct to Magicoder-OSS-

Data-Selection Method	HumanEval (+)
Random Selection	72.6 (68.3)
Textual Score	73.8 (69.5)
Lowest Perplexity	70.1 (67.7)
Highest Perplexity	70.7 (67.7)
YES Pseudo-probability (ours)	**76.2 (72.0)**

Table 8: **Comparison of our data selection method with alternatives (for CL-7B).**

Selected Instructions	HumanEval (+)	MBPP (+)
Top-1 (ours)	**76.2 (72.0)**	**70.6 (60.1)**
Top-3	70.1 (67.1)	68.0 (58.5)
Top-5	70.1 (65.2)	61.9 (53.4)

Table 9: **Performance comparison of the models (CL-7B) trained with different numbers of selected instructions.** "Top-k" means that for each response, we select the instructions with the top k highest pseudo-probability.

Instruct-75K (Wei et al. 2023), a smaller dataset generated by GPT-3.5. The results (Table 7) show that performance is still improved even with a smaller and lower-quality original dataset, demonstrating the robustness of Inverse-Instruct.

6.5 Alternative Data Selection Methods

Our data selection method outperforms alternatives. We compare our data selection method which is based on the pseudo-probability of `YES` with the three alternatives:

1. Randomly selecting one instruction from all synthetic candidates corresponding to each response.
2. Using textual format scores (1-5) provided by the LLM itself as an indicator. If no textual score is given, assign a default score of 3.
3. Using the sentence perplexity of the response code under different instructions as an indicator. We select the data with the highest and lowest perplexity respectively.

The results are shown in Table 8, demonstrating the pseudo-probability method's efficiency.

6.6 Selecting Multiple Self-Generated Instructions

Selecting multiple self-generated instructions for a single response will harm the model's performance. We select the top-k scoring instructions for each response. The results in Table 9 indicate that the model's performance declines as the number of selected instructions increases. This suggests that open-source code LLMs are not capable of generating a large number of correct instructions, which is why we only select the best instructions in our method.

6.7 Multi-Round Optimization for InverseCoder

Repeatedly applying Inverse-Instruct to InverseCoder does not significantly improve performance. We replace the original model with InverseCoder in the pipeline of Inverse-Instruct and train a new model with the data generated by InverseCoder. The performance results (Table 10) show no significant improvement, which confirms the phenomenon of model collapse caused by repeatedly training on self-generated data (Shumailov et al. 2024).

Model	HumanEval (+)	MBPP (+)
InverseCoder-CL-7B	**76.2** (**72.0**)	**70.6** (60.1)
InverseCoder-CL-7B-V2	75.0 (70.1)	**70.6** (**60.6**)

Table 10: **Performance diffenernce when applying Inverse-Instruct to InverseCoder again.** "V2" means models trained with the data generated by InverseCoder.

Data-Generation Method	HumanEval (+)	MBPP (+)
Code → NL (ours)	**76.2** (**72.0**)	70.6 (60.1)
Code → NL → Code	73.2 (68.9)	67.7 (57.7)
Code → Code → NL	73.2 (68.3)	**70.9** (**62.2**)

Table 11: **Comparison of Inverse-Instruct with other alternative data generation methods which prompt the original model to generate additional code (for CL-7B).**

6.8 Training with Additional Self-Generated Code

Performance cannot be steadily improved when the model is trained with both self-generated instructions and code. We conduct the following two experiments to examine whether training with the code generated by the original model provides additional benefits.

1. **Code → NL → Code:** Regenerating new response code for the new instructions obtained by Inverse-Instruct.
2. **Code → Code → NL:** Prompting the original model to give more complex code and applying Inverse-Instruct to the new code.

The results are shown in Table 11. Unstable performance reveals issues with the quality of the self-generated code of original models.

7 Conclusion

In conclusion, this paper presents a novel approach to enhancing the capabilities of open-source code LLMs by leveraging self-generated data for instruction tuning, rather than relying solely on data from powerful closed-source LLMs like GPT-3.5 and GPT-4. Our proposed method, named Inverse-Instruct, capitalizes on the inherent asymmetry in translating between formal and informal languages. By reversing the conventional process, Inverse-Instruct generates additional natural language instructions from code snippets via summarization and self-evaluation techniques. The effectiveness of this methodology is demonstrated through the development of InverseCoder, a new series of code LLMs that not only outperform their predecessors in traditional benchmarks but also show significant improvement across diverse coding tasks.

Acknowledgements

We thank Lei Qi for helping us analyze data and convert NL to code in sanity check experiments (Section 3) during the rebuttal.

This work is partially supported by the National Key R&D Program of China (under Grant 2022YFB4501600), the NSF of China (under Grants 61925208, U22A2028, 6240073476, 62222214, 62341411, 62102398, 62102399, 62302478, 62302482, 62302483, 62302480,62302481), Strategic Priority Research Program of the Chinese Academy of Sciences, (Grant No.XDB0660200, XDB0660201, XDB0660202), CAS Project for Young Scientists in Basic Research (YSBR-029), Youth Innovation Promotion Association CAS and Xplore Prize.

References

Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Bai, J.; Bai, S.; Chu, Y.; Cui, Z.; Dang, K.; Deng, X.; Fan, Y.; Ge, W.; Han, Y.; Huang, F.; Hui, B.; Ji, L.; Li, M.; Lin, J.; Lin, R.; Liu, D.; Liu, G.; Lu, C.; Lu, K.; Ma, J.; Men, R.; Ren, X.; Ren, X.; Tan, C.; Tan, S.; Tu, J.; Wang, P.; Wang, S.; Wang, W.; Wu, S.; Xu, B.; Xu, J.; Yang, A.; Yang, H.; Yang, J.; Yang, S.; Yao, Y.; Yu, B.; Yuan, H.; Yuan, Z.; Zhang, J.; Zhang, X.; Zhang, Y.; Zhang, Z.; Zhou, C.; Zhou, J.; Zhou, X.; and Zhu, T. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609*.

Broder, A. 1997. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*, 21–29.

Cassano, F.; Gouwar, J.; Nguyen, D.; Nguyen, S. D.; Phipps-Costin, L.; Pinckney, D.; Yee, M.-H.; Zi, Y.; Anderson, C. J.; Feldman, M. Q.; Guha, A.; Greenberg, M.; and Jangda, A. 2023. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Transactions on Software Engineering*, 49: 3675–3691.

Chaudhary, S. 2023. Code Alpaca: An Instruction-following LLaMA model for code generation. https://github.com/sahil280114/codealpaca.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chowdhery, A.; Narang, S.; Devlin, J.; Bosma, M.; Mishra, G.; Roberts, A.; Barham, P.; Chung, H. W.; Sutton, C.; Gehrmann, S.; Schuh, P.; Shi, K.; Tsvyashchenko, S.; Maynez, J.; Rao, A.; Barnes, P.; Tay, Y.; Shazeer, N. M.; Prabhakaran, V.; Reif, E.; Du, N.; Hutchinson, B. C.; Pope, R.; Bradbury, J.; Austin, J.; Isard, M.; Gur-Ari, G.; Yin, P.; Duke, T.; Levskaya, A.; Ghemawat, S.; Dev, S.; Michalewski, H.; García, X.; Misra, V.; Robinson, K.; Fedus, L.; Zhou, D.; Ippolito, D.; Luan, D.; Lim, H.; Zoph, B.; Spiridonov, A.; Sepassi, R.; Dohan, D.; Agrawal, S.; Omernick, M.; Dai, A. M.; Pillai, T. S.; Pellat, M.; Lewkowycz, A.; Moreira, E.; Child, R.; Polozov, O.; Lee, K.; Zhou, Z.; Wang, X.; Saeta, B.; Díaz, M.; Firat, O.; Catasta, M.; Wei, J.; Meier-Hellstern, K. S.; Eck, D.; Dean, J.; Petrov, S.; and Fiedel, N. 2022. PaLM: Scaling Language Modeling with Pathways. *J. Mach. Learn. Res.*, 24: 240:1–240:113.

Guo, D.; Zhu, Q.; Yang, D.; Xie, Z.; Dong, K.; Zhang, W.; Chen, G.; Bi, X.; Wu, Y.; Li, Y. K.; Luo, F.; Xiong, Y.; and Liang, W. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *ArXiv*, abs/2401.14196.

Gupta, T.; and Kembhavi, A. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 14953–14962.

Irugalbandara, C.; Mahendra, A.; Daynauth, R.; Arachchige, T. K.; Flautner, K.; Tang, L.; Kang, Y.; and Mars, J. 2023. Scaling Down to Scale Up: A Cost-Benefit Analysis of Replacing OpenAI's GPT-4 with Self-Hosted Open Source SLMs in Production. *ArXiv*, abs/2312.14972.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*.

Lai, Y.; Li, C.; Wang, Y.; Zhang, T.; Zhong, R.; Zettlemoyer, L.; Yih, W.-t.; Fried, D.; Wang, S.; and Yu, T. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, 18319–18345. PMLR.

Li, R.; Allal, L. B.; Zi, Y.; Muennighoff, N.; Kocetkov, D.; Mou, C.; Marone, M.; Akiki, C.; Li, J.; Chim, J.; Liu, Q.; Zheltonozhskii, E.; Zhuo, T. Y.; Wang, T.; Dehaene, O.; Davaadorj, M.; Lamy-Poirier, J.; Monteiro, J.; Shliazhko, O.; Gontier, N.; Meade, N.; Zebaze, A.; Yee, M.-H.; Umapathi, L. K.; Zhu, J.; Lipkin, B.; Oblokulov, M.; Wang, Z.; Murthy, R.; Stillerman, J.; Patel, S. S.; Abulkhanov, D.; Zocca, M.; Dey, M.; Zhang, Z.; Fahmy, N.; Bhattacharyya, U.; Yu, W.; Singh, S.; Luccioni, S.; Villegas, P.; Kunakov, M.; Zhdanov, F.; Romero, M.; Lee, T.; Timor, N.; Ding, J.; Schlesinger, C.; Schoelkopf, H.; Ebert, J.; Dao, T.; Mishra, M.; Gu, A.; Robinson, J.; Anderson, C. J.; Dolan-Gavitt, B.; Contractor, D.; Reddy, S.; Fried, D.; Bahdanau, D.; Jernite, Y.; Ferrandis, C. M.; Hughes, S. M.; Wolf, T.; Guha, A.; von Werra, L.; and de Vries, H. 2023. StarCoder: may the source be with you! *ArXiv*, abs/2305.06161.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Tom; Eccles; Keeling, J.; Gimeno, F.; Lago, A. D.; Hubert, T.; Choy, P.; de, C.; d'Autume, M.; Babuschkin, I.; Chen, X.; Huang, P.-S.; Welbl, J.; Gowal, S.; Alexey; Cherepanov; Molloy, J.; Mankowitz, D. J.; Robson, E. S.; Kohli, P.; de, N.; Freitas; Kavukcuoglu, K.; and Vinyals, O. 2022. Competition-level code generation with AlphaCode. *Science*, 378: 1092 – 1097.

Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–9500. IEEE.

Liu, J.; Xia, C.; Wang, Y.; and Zhang, L. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *ArXiv*, abs/2305.01210.

Lozhkov, A.; Li, R.; Allal, L. B.; Cassano, F.; Lamy-Poirier, J.; Tazi, N.; Tang, A.; Pykhtar, D.; Liu, J.; Wei, Y.; et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173*.

Luo, Z.; Xu, C.; Zhao, P.; Sun, Q.; Geng, X.; Hu, W.; Tao, C.; Ma, J.; Lin, Q.; and Jiang, D. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*.

Ma, Y. J.; Liang, W.; Wang, G.; Huang, D.-A.; Bastani, O.; Jayaraman, D.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*.

Microsoft. 2023. GitHub Copilot – Your AI pair programmer. https://github.com/features/copilot.

Muennighoff, N.; Liu, Q.; Liu, Q.; Zebaze, A.; Zheng, Q.; Hui, B.; Zhuo, T. Y.; Singh, S.; Tang, X.; von Werra, L.; and Longpre, S. 2023. OctoPack: Instruction Tuning Code Large Language Models. *ArXiv*, abs/2308.07124.

Nejjar, M.; Zacharias, L.; Stiehle, F.; and Weber, I. 2023. LLMs for Science: Usage for Code Generation and Data Analysis. *arXiv preprint arXiv:2311.16733*.

Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; and Xiong, C. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

OpenAI. 2022. Chatgpt: Optimizing language models for dialogue.

OpenAI, R. 2023. GPT-4 technical report. arXiv 2303.08774. *View in Article*.

Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140): 1–67.

Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.; Adi, Y.; Liu, J.; Remez, T.; Rapin, J.; Kozhevnikov, A.; Evtimov, I.; Bitton, J.; Bhatt, M. P.; Ferrer, C. C.; Grattafiori, A.; Xiong, W.; D'efossez, A.; Copet, J.; Azhar, F.; Touvron, H.; Martin, L.; Usunier, N.; Scialom, T.; and Synnaeve, G. 2023. Code Llama: Open Foundation Models for Code. *ArXiv*, abs/2308.12950.

Shazeer, N. M.; and Stern, M. 2018. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. *ArXiv*, abs/1804.04235.

Shumailov, I.; Shumaylov, Z.; Zhao, Y.; Papernot, N.; Anderson, R. J.; and Gal, Y. 2024. AI models collapse when trained on recursively generated data. *Nat.*, 631(8022): 755–759.

Shypula, A.; Madaan, A.; Zeng, Y.; Alon, U.; Gardner, J. R.; Yang, Y.; Hashemi, M.; Neubig, G.; Ranganathan, P.; Bastani, O.; and Yazdanbakhsh, A. 2024. Learning Performance-Improving Code Edits. In *The Twelfth International Conference on Learning Representations*.

Song, Z.; Wang, Y.; Zhang, W.; Liu, K.; Lyu, C.; Song, D.; Guo, Q.; Yan, H.; Lin, D.; Chen, K.; and Zhao, C. 2024. AlchemistCoder: Harmonizing and Eliciting Code Capability by Hindsight Tuning on Multi-source Data. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Sparck Jones, K. 1988. *A statistical interpretation of term specificity and its application in retrieval*, 132–142. GBR: Taylor Graham Publishing. ISBN 0947568212.

Su, H.; Shi, W.; Kasai, J.; Wang, Y.; Hu, Y.; Ostendorf, M.; tau Yih, W.; Smith, N. A.; Zettlemoyer, L.; and Yu, T. 2022. One Embedder, Any Task: Instruction-Finetuned Text Embeddings. *ArXiv*, abs/2212.09741.

Surís, D.; Menon, S.; and Vondrick, C. 2023. Vipergpt: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 11888–11898.

Tang, H.; Key, D.; and Ellis, K. 2024. WorldCoder, a Model-Based LLM Agent: Building World Models by Writing Code and Interacting with the Environment. *arXiv preprint arXiv:2402.12275*.

Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

theblackcat102. 2023. The evolved code alpaca dataset. https://huggingface.co/datasets/theblackcat102/evol-codealpaca-v1.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Wang, J.; Zhang, B.; Du, Q.; Zhang, J.; and Chu, D. 2024a. A Survey on Data Selection for LLM Instruction Tuning. *ArXiv*, abs/2402.05123.

Wang, Y.; He, K.; Dong, G.; Wang, P.; Zeng, W.; Diao, M.; Xu, W.; Wang, J.; Zhang, M.; and Cai, X. 2024b. DolphCoder: Echo-Locating Code Large Language Models with Diverse and Multi-Objective Instruction Tuning. In Ku, L.-W.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4706–4721. Bangkok, Thailand: Association for Computational Linguistics.

Wang, Y.; Kordi, Y.; Mishra, S.; Liu, A.; Smith, N. A.; Khashabi, D.; and Hajishirzi, H. 2022. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*.

Wang, Y.; Wang, W.; Joty, S.; and Hoi, S. C. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *EMNLP*.

Wei, J.; Bosma, M.; Zhao, V. Y.; Guu, K.; Yu, A. W.; Lester, B.; Du, N.; Dai, A. M.; and Le, Q. V. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.

Wei, Y.; Cassano, F.; Liu, J.; Ding, Y.; Jain, N.; de Vries, H.; von Werra, L.; Guha, A.; and Zhang, L. 2024. StarCoder2-Instruct: Fully Transparent and Permissive Self-Alignment for Code Generation.

Wei, Y.; Wang, Z.; Liu, J.; Ding, Y.; and Zhang, L. 2023. Magicoder: Source code is all you need. *arXiv preprint arXiv:2312.02120*.

Xu, C.; Sun, Q.; Zheng, K.; Geng, X.; Zhao, P.; Feng, J.; Tao, C.; and Jiang, D. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Yu, Z.; Zhang, X.; Shang, N.; Huang, Y.; Xu, C.; Zhao, Y.; Hu, W.; and Yin, Q. 2023. Wavecoder: Widespread and versatile enhanced instruction tuning with refined data generation. *arXiv preprint arXiv:2312.14187*.

Zhang, Y.; Luo, Y.; Yuan, Y.; and Yao, A. C.-C. 2024. AutoMathText: Autonomous Data Selection with Language Models for Mathematical Texts. *arXiv preprint arXiv:2402.07625*.

Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Wang, Z.; Shen, L.; Wang, A.; Li, Y.; Su, T.; Yang, Z.; and Tang, J. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X. *ArXiv*, abs/2303.17568.

Zhu, E.; Markovtsev, V.; Astafiev, A.; Khan, A.; Ha, C.; Łukasiewicz, W.; Foster, A.; Sinusoidal36; Thakur, S.; Ortolani, S.; Titusz; Letal, V.; Bentley, Z.; fpug; hguhlich; long2ice; oisincar; Assa, R.; Ibraimoski, S.; Kumar, R.; Tian-Huan, Q.; Rosenthal, M. J.; Joshi, K.; Mann, K.; JonR; Halliwell, J.; and Oriekhov, A. 2024. *ekzhu/datasketch: v1.6.5*. Zenodo.

A Limitations

Our research is subject to two primary limitations. Firstly, Inverse-Instruct may perform worse on models with weaker Code-to-NL ability because the improvement of our method stems from the advantage of Code-to-NL over NL-to-Code. A promising avenue for future exploration is to investigate the disparities in the model's capabilities across different modalities (e.g., Code-to-Code, Code-to-NL, NL-to-Code), which may enable the development of more advanced language models that surpass current performance ceilings. Secondly, synthetic data generation relies on access to high-quality code snippet datasets. Future work should focus on reducing the required code snippet volume to enhance efficiency and feasibility.

B Method Details

Algorithm Workflow. The algorithm workflow of Inverse-Instruct is shown in Algorithm 1 (see Page 15).

Prompts. We show the prompts of Inverse-Instruct for code summarization, self-evaluation, and instruction-tuning in Figure 3 (see Page 15).

Examples. Figure 4 and Figure 5 are examples for the details in Inverse-Instruct. Figure 4 is an example of a response with multiple code snippets and Figure 5 is an example of a summarization mistake.

C Further Analysis

We conducted several further analyses of Inverse-Instruct including:

- The dataset's statistical characteristics (Appendix C.1).
- The way to quantify the mismatch between an LLM's code generation and code summarization ability (Appendix C.2).
- Self-improvement experiments using only base models (Appendix C.4).
- The comparison between Inverse-Instruct and StarCoder2-Instruct (Section C.3).
- An explanation for the different training settings in baselines, and the performance of alternative training orders (Appendix C.5).
- Diversity analysis for instructions generated by Inverse-Instruct (Appendix C.6).

C.1 Dataset Analysis

Following Magicoder (Wei et al. 2023), we conduct further analysis for the datasets.

Categories of Instructions. We use the text embeddings generated by INSTRUCTOR (Su et al. 2022) to analyze categories of the instructions generated by Inverse-Instruct. We calculate the ratios of 10 coding-related categories of the instructions in `evol-codealpaca-v1` and the dataset generated by WizardCoder-GPT4-CL-7B. The results are illustrated in Figure 6 and Figure 7 (see Page 16).

Here is a function which takes a string and a number as arguments. The string is the one you want to split and the number is the number of equal parts you want to split the string into.

```python
def split_string(string, part_count):
    str_len = len(string)
    part_len = str_len // part_count
    parts = []
    for i in range(0, str_len, part_len):
        part = string[i: i + part_len]
        parts.append(part)
    return parts
```

You can call this function with your test string and desired number of parts like this:

```python
test_str = 'This_is_a_test'
print(split_string(test_str, 4))
```

Figure 4: An example response with multiple parts of code.

Good instruction

Design a ruby script to copy content of one tar file (gzip format) into another. The files can be large so they should be treated as binary streams using `binread` and `binwrite`. The paths may vary

Bad instruction

Develop a ruby program which reads from a .tar.gz file (gzipped tar archive), decompresses it and writes the content back into a new .tar.gz file.

Response

```ruby
# read all
contents = File.binread("path-to-file.tar.gz")

# write all
File.binwrite("test.tar.gz", contents)
```

Figure 5: An example of a summarization mistake.

Length Distribution of the Datasets. We depict the length distribution by counting the token for the instructions and responses in `evol-codealpaca-v1` and the dataset with instructions generated by WizardCoder-GPT4-CL-7B. The distributions are shown in Figure 8 and Figure 9. It is noticed that the responses in the new dataset are much shorter since they are pure code snippets extracted from the original dataset.

Similarity with HumanEval. We compute the cosine similarity between the HumanEval benchmark and our datasets using TF-IDF (Sparck Jones 1988) embeddings. The results are shown in Figure 10, which demonstrates that the datasets have a low similarity to the benchmark.

C.2 Self-Consistency on Generation and Summarization

We attempt to quantify the mismatch between the LLM's code generation and code summarization ability by measuring *self-consistency*. That is, whether the model can generate equivalent code after summarizing the code it generates before. Self-consistency is defined as

$$\mathbb{1}(M(M^{-1}(M(x))) = M(x)),$$

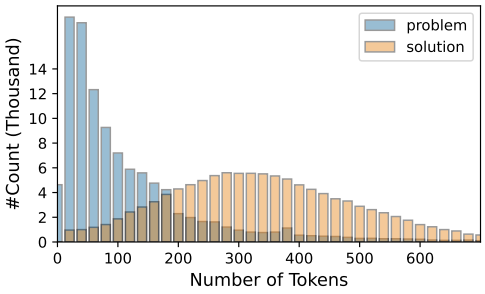

Figure 8: Token count distribution of the instructions and responses in original dataset `evol-codealpaca-v1`.

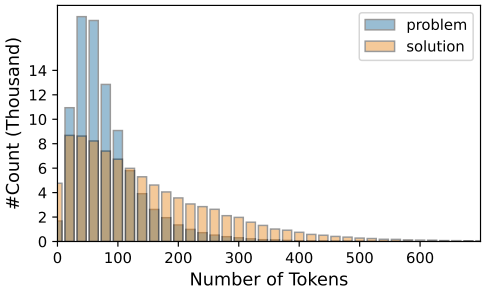

Figure 9: Token count distribution of the instructions and responses in the dataset generated by WizardCoder-GPT4-CL-7B.

where $M(\cdot)$ denotes the LLM's code generation process, $M^{-1}(\cdot)$ denotes the LLM's code summarization process, $\mathbb{1}(\cdot)$ is the indicator function, and $=$ denotes functional equivalence.

Specifically, we use the benchmark MBPP+ to measure the self-consistency in three steps: Firstly, we prompt the code LLM M with problems x_i to generate code $y_i = M(x_i)$. Then, code LLM summarizes the code y_i to get new instructions $x'_i = M^{-1}(y_i)$. Next, we let the code LLM generate new code y'_i based on x'_i, *i.e.* $y'_i = M(x'_i)$. Finally, we evaluate if y'_i and y_i are functional equivalents by measuring their outputs given the same inputs taken from the benchmark. The result is calculated as the pass@1 of new code responses y'_i taking the original code responses y_i as ground truth solutions.

The results are shown in Table 12. InverseCoder has better self-consistency than the original models, which indicates that the original models have a larger gap between generation and summarization, and the performance improvement of InverseCoder may come from bridging this gap.

C.3 Comparison with StarCoder2-Instruct

Inverse-Instruct surpasses another self-improvement method: StarCoder2-Instruct (Wei et al. 2024). Most recently, StarCoder2-Instruct has been introduced as a self-improvement method specifically designed for code LLMs. It uses a similar pipeline to Magicoder (Wei et al. 2023) but

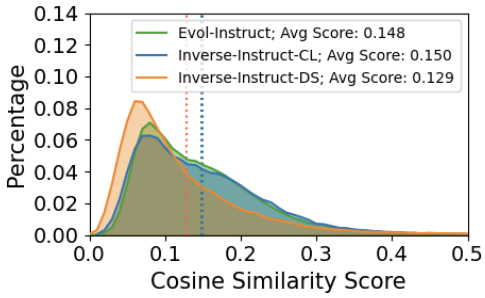

Figure 10: Cosine similarities between HumanEval and different datasets. "Evol-Instruct" represents the original dataset `evol-codealpaca-v1`.

Model	Self-Consistency
WizardCoder-GPT4-CL	69.0 (65.6)
InverseCoder-CL (ours)	**76.0 (73.5)**
WizardCoder-GPT4-DS	76.1 (69.7)
InverseCoder-DS (ours)	**80.4 (75.3)**

Table 12: Pass@1 (%) results on MBPP(+) in the self-consistency experiment. We remove the problems whose inputs will cause a runtime error when applied to the original code response y_i.

with the additional step of generating concepts from code before creating problems. The detailed comparison between Inverse-Instruct and StarCoder2-Instruct is shown in Table 13. We find that Inverse-Instruct takes less time and produces a better instruction-tuned model.

Model	HumanEval (+)	MBPP(+)	Time Cost (h)
Starcoder2-Instruct	71.3 (66.5)	69.6 (59.8)	80
InverseCoder (ours)	**76.2 (72.0)**	**70.6 (60.1)**	**4**

Table 13: **The comparison between our method with Starcoder2-Instruct.** We reproduce it using the same base model (CodeLlama-Python-7B), dataset (we use the response code as seed) and training settings as ours.

C.4 Self-improving for Base Models

Base models can improve themselves by generating instructions from source code through Inverse-Instruct. We replace the instruction-tuned code LLM with its base model in our pipeline to demonstrate that a base model can self-improve solely by relying on source code. Specifically, we use the preprocessed code responses $\{y_i^*\}$ as the unlabeled source code. Then, we apply code summarization and self-evaluation to it using a base model. Finally, we fine-tune the base model only with the instruction data generated by itself. The enhanced performance of the base model (Table 14) reveals that it is useful to apply Inverse-Instruct for the

base model in a situation with adequate unlabeled data but insufficient human-annotated instructions.

Model	HumanEval (+)	MBPP (+)
CodeLlama-Python-7B	39.6 (35.4)	51.9 (43.7)
+1 epoch self-generated	54.3 (49.3)	52.9 (44.7)
+2 epoch self-generated	**54.9 (50.6)**	**54.2 (46.6)**

Table 14: Pass@1 (%) results on HumanEval (+) and MBPP (+) in self-improving experiments for base model CodeLlama-Python-7B. We report performances of the model finetuned with generated data for 1 and 2 epochs.

C.5 Explanation for Different Training Settings Between InverseCoder and Baselines

It is fair to compare InverseCoder with baselines in different training settings. Since the different training settings (epochs, orders for datasets) are also used in previous works (see Table 15), we follow them to report the best results among different training settings in our paper. To validate this fact, we also train Magicoder$\mathcal{S}$-CL in our epoch setting (only train 1 epoch for the lower-quality dataset Magicoder-OSS-Instruct-75K). The results (Table 16) show that the model's performance dropped in our setting.

Model	Training Settings
WizardCoder	200 steps for all data
MagicoderS	2 epochs weak + 2 epochs strong
WaveCoder-Ultra	3 epochs for all data
OpenCodeInterpreter	3 epochs for all data
InverseCoder (ours)	1 epoch weak + 2 epochs strong

Table 15: **Training settings of different instruction-tuned code LLMs.**

Method	HumanEval (+)
2 epochs weak + 2 epochs strong (original)	70.7 (66.5)
1 epochs weak + 2 epochs strong	68.3 (64.6)

Table 16: **Performance degradation after training Magicoder$\mathcal{S}$-CL with our training epoch setting.**

We explore different settings and find that the strong-after-weak order performs the best among three settings. We use the training order of Magicoder$\mathcal{S}$ (Wei et al. 2023), which starts with data generated by a weaker model and then uses data generated by a stronger model. We show the performance results of other training orders in Table 17 to demonstrate our training order's effectiveness.

C.6 Diversity Analysis for Generated Instructions

To compare the diversity of the data generated by Code-to-NL with that generated by NL-to-Code, we reproduce the Diverse Instruction Tuning (DIT) method of DolphCoder (Wang et al. 2024b), which uses different system prompts to obtain diverse code responses. The results (Table 18) show that the performance of DIT is worse than Inverse-Instruct. We believe that it is because the Code-to-NL method generates more diverse data, and we support this idea in two aspects:

Training Order	HumanEval (+)
GPT-4 data (baseline)	72.6 (68.9)
The mixture of GPT-4 and self-generated data	72.6 (64.6)
GPT-4 data + self-generated data	64.0 (59.1)
Self-generated data + GPT-4 data (ours)	**76.2 (72.0)**

Table 17: **The comparison between our training order and other alternatives.** We use CodeLlama-Python-7B as the base model and train for a total of 3 epochs under each order.

Method	HumanEval (+)	MBPP (+)
DIT	69.5 (66.5)	69.8 (58.7)
Inverse-Instruct (ours)	**76.2 (72.0)**	**70.6 (60.1)**

Table 18: **The comparison between Inverse-Instruct and Diverse Instruction Tuning (DIT).** We generate the new code by DIT using WizardCoder-GPT4-CL and `evol-codealpaca-v1`, then apply data selection to it. Other settings are the same as Inverse-Instruct.

The instructions generated by Inverse-Instruct (Code-to-NL) are more diverse than the code generated by DIT (NL-to-Code). We calculate the Jaccard similarity of MinHash (Broder 1997) and cosine similarity of TF-IDF embeddings (Sparck Jones 1988) between each generated instruction and its original one for Inverse-Instruct, and the same similarities between each generated response and its original one for DIT. The results are shown in Figures 11 and 12, which demonstrates that the generated instructions are less similar to the original ones compared to the generated code.

Method	Threshold		
	0.6	0.7	0.8
DIT	8.75	3.53	1.48
Inverse-Instruct (ours)	**7.17**	**1.40**	**0.38**

Table 19: **The overall data duplication rate (%) of Inverse-Instruct and DIT.** We use three different similarity thresholds for MinHash LSH keeping $\texttt{num_perm} = 128$.

Inverse-Instruct tends to produce the datasets with less redundancy. We use MinHash LSH (Zhu et al. 2024) to deduplicate the entire dataset (110K GPT-4 instruction-response pairs and 90K self-generated pairs) for both DIT and Inverse-Instruct. Table 19 is the duplication rate under

different thresholds, which shows that the entire dataset of Inverse-Instruct is more diverse.

D Generation Examples

Tables 20 and 21 (Page 17 and 18) are examples of the responses given by InverseCoder.

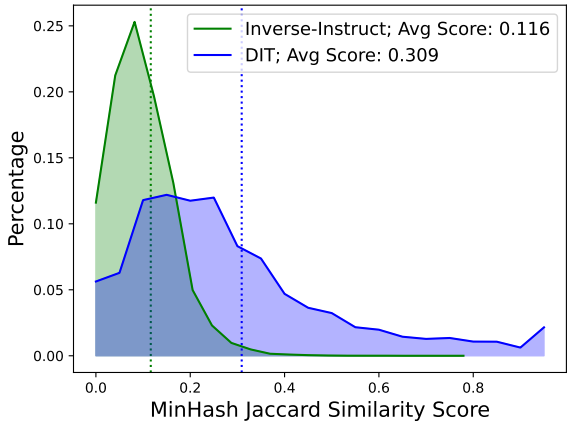

Figure 11: **Inverse-Instruct vs. DIT: MinHash Jaccard similarity between generated data and the original one.**

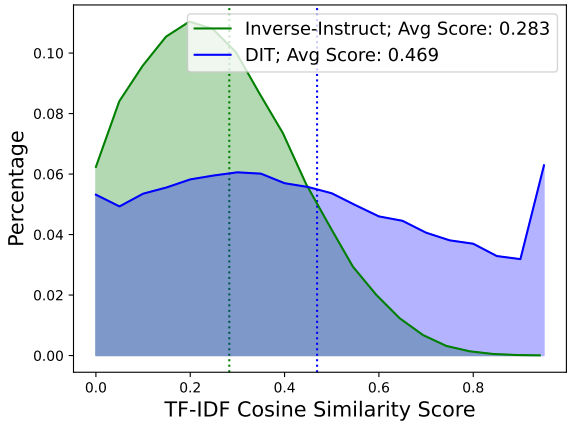

Figure 12: **Inverse-Instruct vs. DIT: TF-IDF cosine similarity between generated data and the original one.**

Algorithm 1: Algorithm Workflow of Inverse-Instruct

Input: An instruction-tuned code LLM M and its instruction-tuning dataset $\mathcal{D}$.
Parameter: The number of summarization samples K. An instruction prefix pool P.
Output: An augmented dataset $\mathcal{D}^*$.

$\mathcal{C} \leftarrow \emptyset$. ▷ Initialize the code corpus
for each instruction-response pair (x_i, y_i) in $\mathcal{D}$ **do** ▷ Preprocess for $\mathcal{D}$
  if there are code snippets in y_i **then**
    $y_i^* \leftarrow$ the first part of the code in y_i.
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{y_i^*\}$.
  end if
end for
$\mathcal{D}^* \leftarrow \emptyset$. ▷ Initialize the augmented dataset
for each code snippet y_i^* in $\mathcal{C}$ **do**
  $\mathcal{I}_i^* \leftarrow \emptyset$ ▷ Initialize the instruction set for y_i^*
  for $j \leftarrow 1$ to K **do**
    $p_j \leftarrow random_choice(P)$ ▷ Randomly select a prefix from the pool P
    $x_{ij}^* \leftarrow M(y_i^*, p_j)$. ▷ Generate a new instruction starting with p_j for y_i^*
    $s_{ij}^* \leftarrow M(x_{ij}^*, y_i^*)$. ▷ Evaluate the correctness score of y_i^* under x_{ij}^*
    $\mathcal{I}_i^* \leftarrow \mathcal{I}_i^* \cup \{(x_{ij}^*, s_{ij}^*)\}$.
  end for
  $(x_i^{**}, s_i^{**}) \leftarrow (x_{ij}^*, s_{ij}^*)$ where $s_{ij}^* = \max_{\mathcal{I}_i^*} s_{ij}^*$. ▷ Select the best instruction in $\mathcal{I}_i^*$
  $\mathcal{D}^* \leftarrow \mathcal{D}^* \cup \{(x_i^{**}, y_i^*)\}$.
end for
return $\mathcal{D}^*$

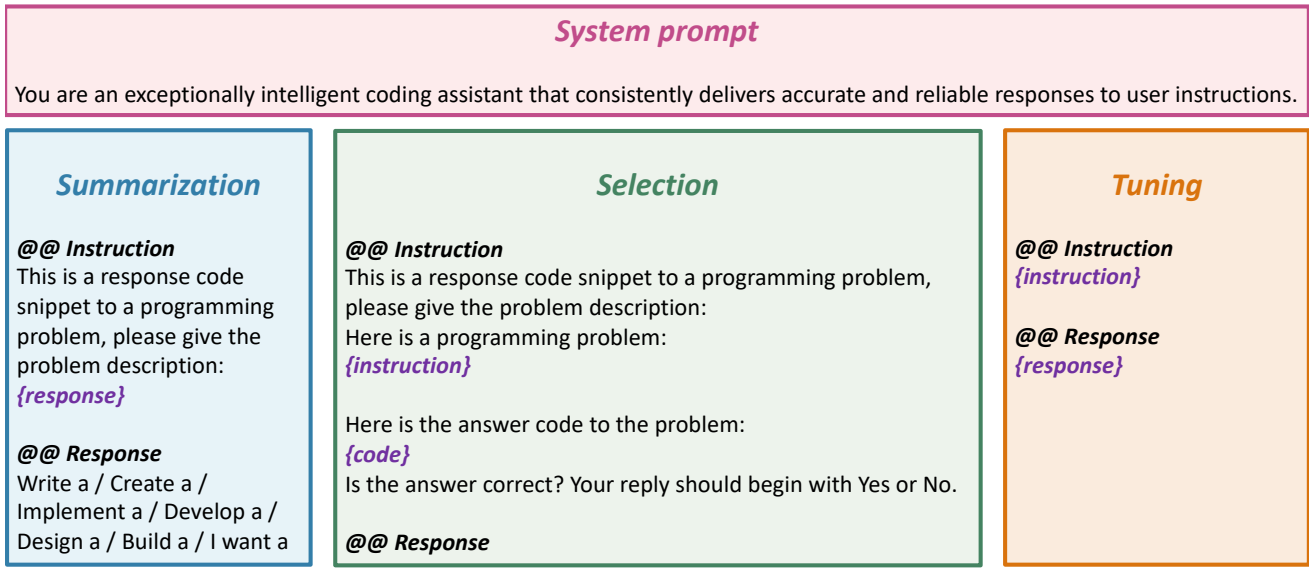

Figure 3: **The prompts of Inverse-Instruct for code summarization, self-evaluation, and instruction-tuning.** For code summarization, we use a diverse set of initial verbs in the prefixes to ensure the overall diversity of the instructions. We first count the first verb frequencies of each instruction in the original dataset and choose the top 5 most frequent verbs. Then we ask ChatGPT to give similar verbs to expand the first verb pool for prompt prefixes.

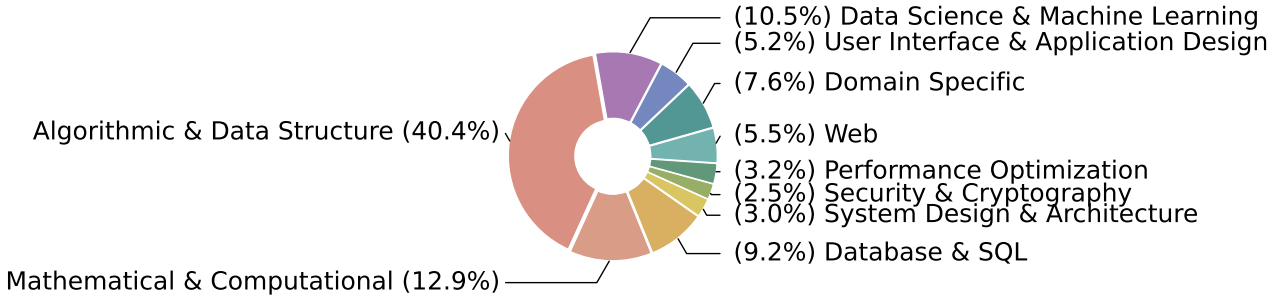

Figure 6: Instruction category constitution of `evol-codealpaca-v1`.

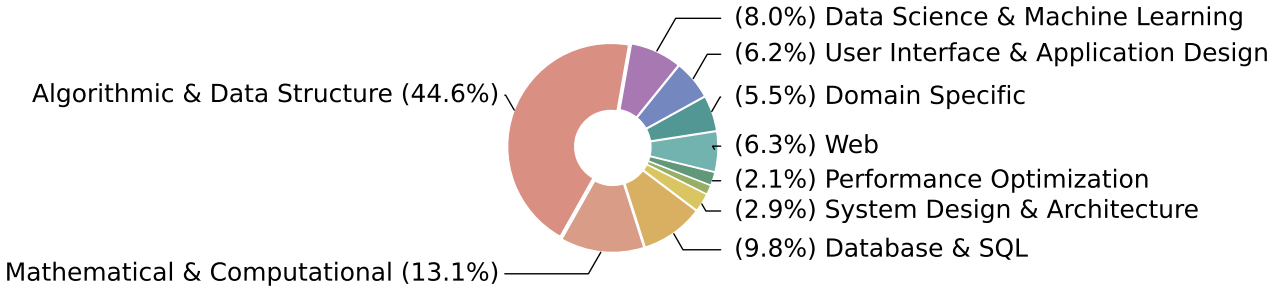

Figure 7: Instruction category constitution of the dataset generated by WizardCoder-GPT4-CL-7B.

<table>
<tr><th>Instruction</th><th>Response</th></tr>
<tr><td>Write a Python script that lists all the files in the current directory.</td><td>Here is a simple Python script that lists all the files in the current directory:
<pre>import os

# Get the current directory
current_dir = os.getcwd()

# List all files in the current directory
files = os.listdir(current_dir)

# Print the files
for file in files:
    print(file)</pre>
This script uses the `os` module in Python, which provides a way of using operating system dependent functionality. The `os.getcwd()` function returns the current working directory, and `os.listdir(path)` returns a list containing the names of the entries in the directory given by path.</td></tr>
<tr><td>Please show me how to calculate the first ten primes in cpp.</td><td>Here is a simple C++ program that calculates the first ten prime numbers:
<pre>#include<iostream>
using namespace std;

bool isPrime(int num) {
    if (num <= 1)
        return false;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0)
            return false;
    }
    return true;
}
int main() {
    int count = 0;
    int num = 2;
    while(count < 10) {
        if(isPrime(num)) {
            cout << num << " ";
            count++;
        }
        num++;
    }
    return 0;
}</pre>
This program defines a function `isPrime` that checks if a number is prime. It then uses this function in a loop in the `main` function to find and print the first ten prime numbers.</td></tr>
</table>

Table 20: Instruction and response examples of InverseCoder.

<table>
<tr><th>Instruction</th><th>Response</th></tr>
<tr><td>How to do matrix multiplication in Pytorch? Show me an example.</td><td>Matrix multiplication in PyTorch can be done using the `mm` function from the `torch` module. Here is an example:
<pre>import torch

# Create two matrices
matrix1 = torch.tensor([[1, 2, 3], [4, 5, 6]])
matrix2 = torch.tensor([[7, 8], [9, 10], [11, 12]])

# Perform matrix multiplication
result = torch.mm(matrix1, matrix2)

print(result)</pre>
In this example, `matrix1` is a 2x3 matrix and `matrix2` is a 3x2 matrix. The `mm` function performs matrix multiplication on these two matrices and returns a new matrix.
The output will be:
<pre>tensor([[58, 64],
        [139, 154]])</pre></td></tr>
<tr><td>Please check if the network can connect to www.example.com using Python.</td><td>Sure, you can use the `requests` library in Python to check if a network can connect to `www.example.com`. Here is a simple script:
<pre>import requests

try:
    response = requests.get("http://www.example.com")
    response.raise_for_status()
    print("Connection successful")
except requests.exceptions.HTTPError as errh:
    print ("Http Error:",errh)
except requests.exceptions.ConnectionError as errc:
    print ("Error Connecting:",errc)
except requests.exceptions.Timeout as errt:
    print ("Timeout Error:",errt)
except requests.exceptions.RequestException as err:
    print ("Something went wrong",err)</pre>
This script sends a GET request to `www.example.com` and checks for any exceptions that might occur during the request. If the request is successful, it prints “Connection successful”. If there is an HTTP error, a connection error, a timeout error, or any other type of request exception, it prints the error.</td></tr>
</table>

Table 21: Instruction and response examples of InverseCoder.